Money Manager Application

1. Introduction

The Money Manager application is a full-stack web application designed to help users track, analyze, and manage their financial activities. It provides secure authentication, efficient data handling, and an intuitive user interface to improve personal finance management.

2. Objectives

- Help users manage daily financial transactions
- Provide analytical insights into spending behavior
- Ensure secure authentication and authorization
- Maintain data consistency and integrity
- Build a scalable and maintainable system

3. Background & Theory

Personal finance management systems are based on principles of **data organization, categorization, and analysis**. The system uses:

- **CRUD Operations** (Create, Read, Update, Delete) for managing transactions
- **Client-Server Architecture** for separation of concerns
- **RESTful APIs** for communication between frontend and backend
- **Authentication & Authorization** using JWT for secure access

4. System Architecture

The application follows a **3-tier architecture**:

1. Presentation Layer (Frontend)

- Built using React.js
- Responsible for UI rendering and user interaction

2. Application Layer (Backend)

- Built using Spring Boot
- Handles business logic, validation, and API processing

3. Data Layer (Database)

- MySQL database
- Stores structured financial data

5.Technologies Used

Frontend

- React.js
- Axios
- Tailwind

Backend

- Spring Boot
- Spring Security (JWT Authentication)
- REST APIs

Database

- MySQL

6. Module Description

6.1 Authentication Module

- User Registration
- Login functionality
- Token-based authentication (JWT)

6.2 Expense Management Module

- Add, update, delete expenses
- Categorize expenses

6.3 Income Management Module

- Add and track income sources

6.4 Dashboard Module

- Displays financial summary
- Provides quick insights

7. Data Flow

1. User enters login credentials
2. Frontend sends request to backend API
3. Backend validates credentials
4. JWT token is generated and returned
5. Token is stored on client-side
6. User performs operations (add/view/edit data)
7. Backend verifies token for each request
8. Database is updated accordingly
9. Response sent back to frontend

8. Database Design

ER Concept

- One user can have many expenses and incomes (1:N relationship)

User :

The **User table** is responsible for storing authentication and profile details of users.

Attributes:

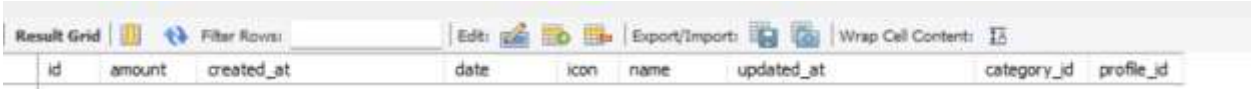
- **id (Primary Key)**
  Unique identifier for each user.
- **email**
  Stores user email (used for login).
- **full_name**
  Stores the name of the user.
- **password**
  Stores encrypted password (using hashing like BCrypt).
- **activation_token**
  Used for account verification and email activation.
- **is_active**
  Boolean field indicating whether the account is activated.
- **created_at**
  Stores account creation timestamp.

Expense

Attributes:

- **id (Primary Key)**
  Unique identifier for each expense.
- **amount**
  Stores the amount spent.
- **name**
  Represents the title of the expense (e.g., Food, Travel).
- **date**
  The actual date when the expense occurred.

- **created_at**
  Timestamp when the record was created.
- **updated_at**
  Timestamp when the record was last updated.
- **icon**
  Stores an icon or visual identifier for the expense category.
- **category_id (Foreign Key)**
  Links the expense to a category (like Food, Bills, etc.).
- **profile_id (Foreign Key)**
  Links the expense to a specific user/profile.



Income

Attributes:

- **id (Primary Key)**
Unique identifier for each transaction.
- **amount**
Represents the monetary value of the transaction.
- **name**
Describes the transaction (e.g., Salary, Food, Rent).
- **date**
The date on which the transaction occurred.
- **created_at**
Timestamp when the transaction was created.
- **updated_at**
Timestamp when the transaction was last modified.
- **icon**
Used for UI representation of transaction type.
- **category_id (Foreign Key)**
Links transaction to a category.
- **profile_id (Foreign Key)**
Associates transaction with a specific user

9.Future Enhancements

- Data visualization using charts
- Budget planning feature
- Mobile application version
- Cloud deployment (AWS/Render)